

| Introdução Ao Java

DCC024

João Eduardo Montandon

| Pensando Em Objetos

Um paradigma de programação dita um estilo de resolução de problemas.

- O paradigma funcional favorece a representação por meio de *funções*.
 - O paradigma orientado a objetos favorece a representação por meio de *objetos*.
-

Mas o que isso quer dizer? 🤔

| Exemplo: Ponto2D

Como mover um ponto na tela?

- **Imperativo:** O computador sabe como mover um ponto na tela.
 - **Funcional:** Uma função sabe como computar a movimentação do ponto.
 - **Objetos:** Um objeto sabe como mover a si mesmo.
-

Em programas orientados a objetos, nós empoderamos as representações de dados (objetos) para que eles *sejam responsáveis pelas suas próprias ações*.

```
public class Point {  
    private int x,y;  
    private Color cor;  
  
    public int getX() {  
        return x;  
    }  
  
    public int getY() {  
        return y;  
    }  
  
    public void move(int newX, int newY) {  
        x = newX;  
        y = newY;  
    }  
}
```

| Por Que Java?

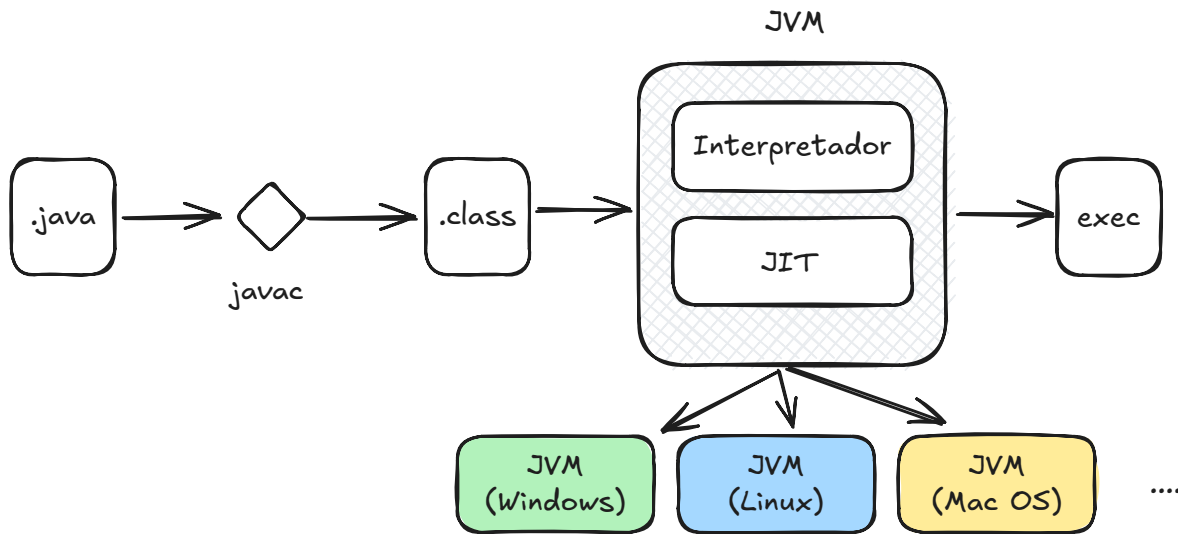
Quais eram os desafios dos programas desenvolvidos nos anos 80 e 90?

1. Portabilidade
2. Segurança e robustez
3. Baixa padronização

Princípios

1. Simples, Orientada a Objetos, e familiar
2. Robusta e Segura
3. Estruturalmente neutra e portátil
4. Alta performance

| JVM: Grande Inovação



Controle total sobre o funcionamento da linguagem, *em qualquer plataforma*.

| Principais Características

1. Expressões
 2. Comandos
 3. Definições de Classes
 4. Referências e Ponteiros
 5. "Hello, Java 2.0"
-

| Expressões

| Tipos Primitivos

```
public class Types {  
    public static void main(String[] args) {  
        short s = 32767; // 16 bits (com sinal)  
        char c = (char)65535; // 16 bits (sem sinal)  
        int i = c; // 32 bits (com sinal)  
        float f = 1.0F; // 32 bits IEEE 754  
        double d = 2.0; // 64 bits IEEE 754  
        byte b = 127; // 8 bits com sinal  
        System.out.println("int = " + i);  
        System.out.println("char = " + c);  
        System.out.println("float = " + f);  
        System.out.println("double = " + d);  
        System.out.println("byte = " + b);  
        System.out.println("short = " + s);  
    }  
}
```

| Tipos Referenciados

Qualquer valor que não é primitivo, é uma referência a um objeto. Há três categorias de referência:

- Classes
- Interfaces
- Arrays

| Strings

Strings são considerados objetos na linguagem, porém com tratamento especial.

```
String nome = "João";  
System.out.println(nome.length());
```

```
System.out.println(nome.charAt(2));  
System.out.println(nome.toUpperCase());
```

| Operadores Aritméticos

Expressão	Valor
<code>1 + 2 * 3</code>	7
<code>15 / 7</code>	2
<code>15.0 / 7.0</code>	2.142857142857143
<code>15 % 7</code>	1

Coerção é feita automaticamente durante as operações aritméticas, mas garantindo a precisão dos tipos envolvidos

Expressão	Valor
<code>1/2</code>	<code>0</code>
<code>1/2.0</code>	<code>0.5</code>
<code>1 * (1.0/2.0)</code>	<code>1.5</code>

| Concatenação (Strings)

Operador `+` é sobrecarregado quando um dos termos envolvidos na operação for uma String.

Expressão	Valor
<code>"123" + "456"</code>	<code>"123456"</code>
<code>"Eu sou o No " + 4</code>	<code>"Eu sou o No 4"</code>

Expressão	Valor
" " + (1.0 / 2.0)	"0.5"
1 + "2"	"12"
"1" + 2 + 3	"123"
1+2+"3"	"33" 🤔

| Comparação

- Resultado produzido é `boolean`
- `<` `>` `<=` `>=`
- Comparações de igualdade: `=` `!=`
 - Tipos Primitivos: por valor.
 - Tipos Referenciados: por posição de memória.

| Casos Especiais

```
public class Operators {  
    public static void main(String[] args) {  
        Integer a = 10;  
        Integer b = a;  
        Integer c = 10;  
  
        System.out.println(a == b);  
        System.out.println(a == c);  
        System.out.println("a" == "a");  
    }  
}
```

O que vai ser impresso aqui? 🤔

| Operadores Booleanos

Expressão	Valor
<code>1 ≤ 2 && 2 ≤ 3</code>	<code>true</code>
<code>1 < 2 1 > 2</code>	<code>true</code>
<code>1 < 2 ? 3 : 4</code>	<code>true</code>

| Operadores Com Efeitos Colaterais

- Um operador possui *efeito colateral* se ele alterar algo no ambiente do programa.
- No paradigma funcional, os operadores são *puros*, sem efeito colateral.
- No paradigma imperativo esses operadores são fundamentais, pois eles são os responsáveis por transformar os dados do programa.

| Atribuição

```
a = 1
```

O que isso significa em ML? 🤔

O que isso significa em Java? 🤔

| Rvalues E Lvalues

Por que `a=1` faz sentido, mas `1=a` não? 🤔

- **Rvalues:** Expressões a direita precisam ter um valor.
- **Lvalues:** Expressões a esquerda precisam referenciar locais de memória.

Operadores Compostos

Expressão	Valor
<code>a=a+b</code>	<code>a+=b</code>
<code>a=a-b</code>	<code>a-=b</code>
<code>a=a*b</code>	<code>a*=b</code>
<code>a=a+1</code>	<code>++a</code>
<code>a=a-1</code>	<code>--a</code>

Expressões Com Efeitos Colaterais

Expressão	Valor	Efeito Colateral
<code>v[++a]</code>	<code>v[a+1]</code>	Altera <code>a</code> antes de acessar <code>v</code>
<code>v[a++]</code>	<code>v[a]</code>	Acessa <code>v</code> , e então altera <code>a</code>
<code>a+(x=b)+c</code>	<code>a+b+c</code>	Altera <code>x</code> para ser igual a <code>b</code>
<code>a=b=c</code>	<code>c</code>	Altera os valores de <code>a</code> e <code>b</code> para <code>c</code>

Como isso é possível? Atribuições retornam o valor atribuído. 🤔

Chamada De Métodos

```
<method-call> ::= <ref-expr>.<method-name>(<param-list>)  
                  | <class-name>.<method-  
name>(<param-list>)  
                  | <method-name>(<param-
```



```
list>)
```

| Chamadas De Instância

- Métodos que irão definir o que uma instância da classe (objeto) poderá fazer.
- Atua diretamente no objeto que chamou o método (*caller*).
- Contexto de execução é provido pelo *caller*.

```
String s = "Linguagens";  
String r = "Programação";  
s.length();  
s.equals(r);  
r.equals(s);  
r.toUpperCase();  
s.toUpperCase().charAt(2);
```

| Chamadas De Classe

- Define o que uma classe propriamente dita poderá fazer.
- A classe serve apenas como um [namespace nomeado](#).
- Funções não fazem parte do paradigma orientado a objetos.

```
String.valueOf(1=2);  
String.valueOf(5*5);  
String.format("%s: %d", nome, idade);
```

| Criação De Objetos

```
<creation-expr> ::= new <class-name> (<param-list>)
```

- **new** irá criar uma nova instância para a classe.
- Quando um novo objeto é criado, seu *construtor* é chamado.
- Uma classe pode ter um ou mais construtores.
- Não é necessário desalocar os objetos (Coletor de lixo).

```
String a = new String();  
String b = new String(a);
```

```
public class Point {  
    ...  
    public Point(int x, int y) {  
        this.x = x;  
        this.y = y;  
    }  
    public Point() {  
        this(0,0);  
    }  
    ...  
}  
...  
Point p1 = new Point(1,2);  
Point p2 = new Point();
```

| Informações Gerais Dos Operadores

- Todos são associados a esquerda, com exceção de atribuições
- 15 níveis de precedência
 - Use parênteses em caso de dúvidas
- Muitas coerções
 - `null` para qualquer tipo referenciado
 - Qualquer valor para `String` em concatenação
 - Boxing e Unboxing (`Integer`, `Double`, `Boolean`, etc)

| Comandos

O que é um comando? 🤔

Comandos *produzem efeitos colaterais* durante sua execução. É dessa forma que linguagens imperativas realizam suas computações.

Comandos Principais

1. Expressões
 2. Desvio de fluxo (Condicionais e repetições)
 3. Sequenciais (Blocos)
 4. Atribuições
-

| Expressões

- Comando único.
- Deve produzir um efeito colateral em sua computação.

```
speed = 0;  
a++;  
negativo = saldo < 0;  
frase.toString();
```

| Declarações

- Introduz uma nova variável ao programa.
- Escopo atrelado ao bloco no qual a variável foi definida.

```
boolean done = false;  
Point p;  
String a = new String("alô");
```

| Blocos

Conjunto de comandos a serem executados e o bloco for ativado.

<pre>{ a = 0; b = 1; }</pre>	Guarda 0 em a, e 1 em b.
<pre>{ a++; b++; c++; }</pre>	Incrementa os valores de a, b, e c.
<pre>{ }</pre>	Não faz nada.

| Desvio De Fluxo

Alteram dinamicamente o fluxo de execução do programa.

- Condicionais: `if-else`, `switch`
- Repetições: `while`, `do-while`, `for`

```
for(int i = 0; i < 20; i++) {  
    if(i % 2 == 0)  
        System.out.println("par");  
}
```

| Comando `return`

```
<return-stmt> ::= return <expression>; | return;
```

- Primeiro caso: quando função explicita um valor como retorno.
- Segundo caso: funções `void`.
- Em ambos, interrompe execução do método.

| Classes

Como escrever um fatorial em C?

```
int fact(int n) {  
    int f = 1;  
    while(n > 0)  
        f = f * n--;  
}
```

```
        return f;
    }
```

Como escrever fatorial em Java?

```
public class Fact {
    public static int fact(int n) {
        int f = 1;
        while(n > 0)
            f = f * n--;
        return f;
    }
}
```

Qual a diferença entre as duas? 🤔

- Lembre-se estamos falando de uma linguagem com ênfase em OO.
- Sua sintaxe incentiva a adoção de práticas alinhadas a esse paradigma.
- Exemplo: funções e variáveis devem pertencer a uma estrutura (e.g., classes).

| Conceitos Básicos

- Uma classe é uma implementação de um tipo.
- Toda vez que uma classe é instanciada, um objeto é produzido.
- Métodos correspondem a ações que podem ser feitas sobre um objeto.
- Atributos determinam o estado em que o objeto se encontra.

| Exemplo: Lista Funcional

```
public class ConsCell {
    private int head; // atributo privado
    private ConsCell tail; // atributo privado

    /** Construtor da Classe */
    public ConsCell(int h, ConsCell t) {
        this.head = h;
        this.tail = t;
    }

    public ConsCell(int v) { this(v, null); }

    /** Getters & Setters */
    public int getHead() { return this.head; }
    public ConsCell getTail() { return this.tail; }

    public static ConsCell cons(int v, ConsCell c) {
        return new ConsCell(v, c);
    }

    /** Calcula o tamanho da lista */
    public static int length(ConsCell c) {
        int len = 0;
        while(c != null) {
            c = c.getTail();
            len++;
        }
        return len;
    }

    /** Método principal */
    public static void main(String[] args) {
        ConsCell a = null;
        ConsCell b = ConsCell.cons(2, a);
        ConsCell c = ConsCell.cons(1, b);
        int l = ConsCell.length(a) +
                ConsCell.length(b) +
```

```
        ConsCell.length(c);  
  
        System.out.println(l);  
    }  
}
```

Como torná-lo mais OO? 🤔

Um objeto deve saber o que fazer com seus dados...

```
public class IntList {  
    private ConsCell first;  
  
    public IntList(ConsCell s) {  
        this.first = s;  
    }  
  
    public IntList cons(int v) {  
        ConsCell newCell = new ConsCell(v, first);  
        return new IntList(newCell);  
    }  
  
    public int length() {  
        int len = 0;  
        for(ConsCell c = first; c != null; c =  
c.getTail())  
            len++;  
        return len;  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        IntList a = new IntList(null);  
        IntList b = a.cons(2);  
        IntList c = b.cons(1);  
        int l = a.length() + b.length() + c.length();  
  
        System.out.println(l);  
    }  
}
```

- Como implementar um método que imprima todo conteúdo da lista?
- Como alterar a lista para que ela suporte qualquer tipo? 🤔

| Objetos & Referências

- Em Java, os tipos são primitivos ou referenciados.
- Todo objeto é um tipo referenciado.
- **Todo tipo referenciado é um ponteiro.**
- Portanto, todas as regras que se aplicam a ponteiros, aplicam a objetos.

```
Point p1 = new Point(2,2);  
Point p2 = p1;  
  
p2.move(5,5);  
System.out.println(p1);  
System.out.println(p2);
```

O que vai ser impresso aqui? 🤔

```
public static void increment(int[] v){
    for(int i = 0; i < v.length; i++){
        v[i]++;
    }
    ...
public static void main(String[] args) {
    int[] v = {0, 1, 2, 3};
    increment(v);
    for(int i = 0; i < v.length; i++) {
        System.out.println(v[i]);
    }
}
```

O que vai ser impresso aqui? 🤔

| Considerações Finais

- Java é uma linguagem que prioriza a criação de programas orientados a objetos.
- Sistema de tipos primitivos bem definido: `int`, `char`, `boolean`, etc.
- Tipos referenciados são ponteiros!
- Sistema de coerção e precedência bem elaborado, por vezes complexo.
- Principais comandos produzem efeitos colaterais.
- API bastante extensa e completa.